

Ex.No.3 – Wireshark

Date:

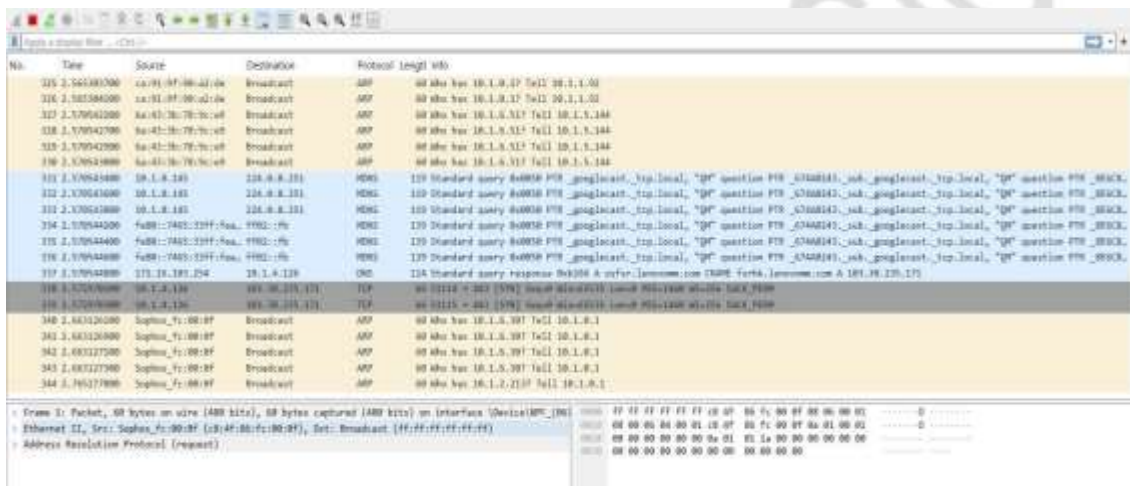
Password Capturing

Wireshark can capture not only passwords but any type of information transmitted over the network: usernames, email addresses, personal information, etc. As long as we can capture network traffic, Wireshark can sniff passing passwords.

In sniffing can include passwords for various protocols such as HTTP, FTP, Telnet, etc. the captured data can be used to troubleshoot network problems, but can also be used maliciously to gain unauthorized access to sensitive information.

So, here we will see how we can capture the password using the Wireshark network capture analyzer. and see the outputs of the following steps.

Step 1: First of all, open your Wireshark tool in your window or in Linux virtual machine. and start capturing the network. suppose I am capturing my wireless fidelity.



Step 2: After starting the packet capturing we will go to the website and login the credential on that website as you can see in the image.

Step 3: Now after completing the login credential we will go and capture the password in Wireshark. for that we have to use some filter that helps to find the login credential through the packet capturing.

Step 4: Wireshark has captured some packets but we specifically looking for HTTP packets. so in the display filter bar we use some command to find all the captured HTTP packets. as you can see in the below image the green bar where we apply the filter.
http

No.	Time	Source	Destination	Protocol	Length	Info
1	2025-06-22 10:14:27	127.0.0.1	127.0.0.1	HTTP	364	GET / HTTP/1.1
3	2025-06-22 10:14:27	127.0.0.1	127.0.0.1	HTTP	404	HTTP/1.1 200 OK (text/html)
5	2025-06-22 10:15:02	127.0.0.1	127.0.0.1	HTTP	447	GET /favicon.ico HTTP/1.1
6	2025-06-22 10:15:02	127.0.0.1	127.0.0.1	HTTP	404	HTTP/1.1 200 OK (text/html)
8	2025-06-22 10:15:35	127.0.0.1	127.0.0.1	HTTP	544	POST /login.php HTTP/1.1 (application/x-www-form-urlencoded)
9	2025-06-22 10:15:36	127.0.0.1	127.0.0.1	HTTP	404	HTTP/1.1 200 OK (text/html)
11	2025-06-22 10:16:10	127.0.0.1	127.0.0.1	HTTP	447	GET /favicon.ico HTTP/1.1
12	2025-06-22 10:16:10	127.0.0.1	127.0.0.1	HTTP	404	HTTP/1.1 200 OK (text/html)

> Frame 707: 76 bytes on wire (608 bits), 76 bytes captured (608 bits) on interface \Device\NPF...
 > Ethernet II, Src: 08:00:00:00:00:00, Dst: 127.0.0.1
 > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
 > Transmission Control Protocol, Src Port: 61231, Dst Port: 80, Seq: 1, Ack: 1, Len: 712
 > Hypertext Transfer Protocol

0000 00 00 53 03 00 00 00 23 20 d0 00 00 00 31 ...K.
 0000 20 00 00 03 00 00 00 00 00 00 00 00 00 00 ...B10.
 0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .../127.1
 0060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...127.1
 0090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...>0001
 00c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...E..
 00f0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...P113
 0120 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...Rpone
 0150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...Rrone
 0180 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...of-562
 01b0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...-B-5-

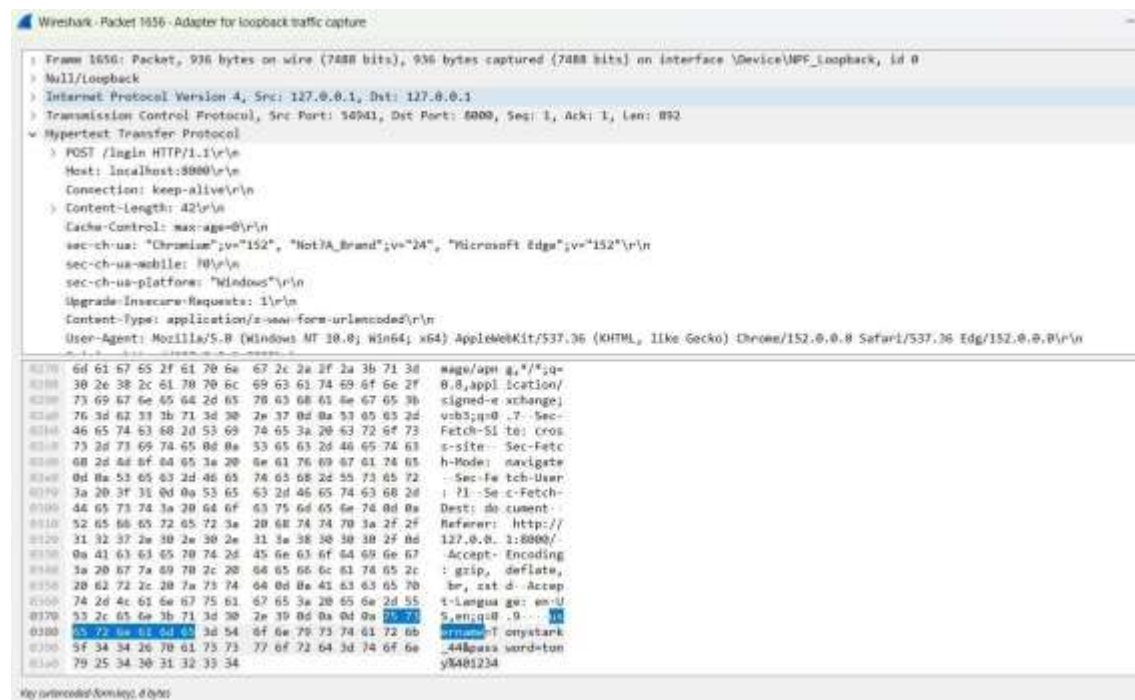
Step 5: So there are some HTTP packets are captured but we specifically looking for form data that the user submitted to the website. for that, we have a separate filter

As we know that there are main two methods used for submitting form data from web pages like login forms to the server. the methods are-

- GET
- POST

Step 6: So firstly for knowing the credential we use the first method and apply the filter for the GET methods as you can see below.

http.request.method == "GET"



GET method

As you can see in the image there are two packets where the login page was requested with a GET request as well, but there is no form data submitted with a GET request.

Step 7: Now after checking the GET method if we didn't find the form data, then we will try the POST method for that we will apply the filter on Wireshark as you can see.

http.request.method == "POST"

